



UNIVERSIDADE ESTADUAL DE SANTA CRUZ
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS
CIÊNCIA DA COMPUTAÇÃO

ANA LUIZA OLIVEIRA DE FREITAS

YURI COUTINHO COSTA

BENCHMARK DE SUAVIZAÇÃO COM OPENMP

Ilhéus - BA

2026

ANA LUIZA OLIVEIRA DE FREITAS

YURI COUTINHO COSTA

BENCHMARK DE SUAVIZAÇÃO COM OPENMP

Relatório apresentado à Faculdade de Ciência da Computação da Universidade Estadual Santa Cruz como requisito parcial para a conclusão da disciplina de Processamento Paralelo, no semestre de 2026.1.

Orientador (a): Prof. Dr. Esbel Tomás Valero Orellana.

Ilhéus - BA

2026

1. INTRODUÇÃO

1.1 Objetivo

Desenvolver e otimizar um algoritmo de suavização/desfoque Gaussiano iterativo em imagens 2D, aplicando técnicas de paralelismo em memória compartilhada (OpenMP).

1.2 Conceitos Essenciais

1.2.1 Suavização/Desfoque Gaussiano

A suavização gaussiana é uma técnica de processamento de imagens que atua como um filtro passa-baixa, reduzindo ruídos e suavizando detalhes por meio de uma média ponderada baseada na distribuição normal. O processo utiliza um kernel quadrado de dimensão ímpar, cujos pesos são ditados pelo desvio padrão/sigma. Enquanto o tamanho do kernel define a área de vizinhança que influenciará o pixel central, o sigma controla a dispersão da "curva de sino" gaussiana; quanto maior o valor de sigma, maior será o peso dado aos pixels vizinhos e, conseqüentemente, mais intenso será o efeito de borrão.

Cada elemento do kernel segue uma função exponencial e, em seguida, é aplicada a normalização para que a soma total de seus pesos seja igual a 1, evitando alterações indesejadas no brilho da imagem. A aplicação prática ocorre via convolução, onde o kernel desliza sobre a imagem original para calcular o novo valor de cada pixel.

1.2.2 Paralelismo e OpenMP

O paralelismo consiste na execução simultânea de múltiplas tarefas computacionais para reduzir o tempo total de processamento, sendo uma abordagem essencial para algoritmos de alta intensidade aritmética. No contexto de sistemas de memória compartilhada, o OpenMP (Open Multi-Processing) destaca-se como a API padrão, operando através do modelo Fork-Join. Nesse modelo, a execução do programa inicia-se em uma única thread mestra que, ao encontrar uma diretiva de paralelização, cria um grupo de threads (fork) para processar dados simultaneamente, sincronizando-as ao final da tarefa (join) para retornar ao fluxo sequencial.

1.3 Importância da análise de desempenho

A análise de desempenho é a etapa que valida a eficácia da paralelização, transformando dados de tempo em métricas quantificáveis. Em processamento paralelo, essa análise permite determinar o *Speedup*, que mede o ganho de velocidade obtido ao distribuir a carga de trabalho entre as múltiplas *threads* do processador em relação à execução puramente sequencial. Além disso, é possível observar a Eficiência do sistema, indicando o quão bem os recursos do hardware, como os núcleos de performance e processador, estão sendo aproveitados para resolver o problema proposto.

2. METODOLOGIA

2.1 Descrição da implementação sequencial

Foram utilizadas 3 funções principais para solucionar o problema:

1. void create_gaussian_kernel() para gerar a máscara que será aplicada na imagem;
2. void apply_convolution() para fazer o padding da imagem e aplicar a máscara nela;
3. double** iterative_gaussian_blur() para aplicar o filtro de gauss iterativamente.

2.2 Descrição da implementação paralela (quais funções OpenMPI foram usadas e por quê)

Durante os testes realizados, foram utilizadas as funções:

1. int omp_get_thread_num() para visualizar o identificador da thread;
2. int omp_get_num_threads() para observar o número de threads que estavam executando no momento;
3. void omp_set_num_threads(int num_threads) para indicar o número de threads a executar na região paralela.

Na implementação paralela foi utilizada a diretiva #pragma omp for, que define a divisão das iterações do laço for, com a cláusula schedule - dynamic, que divide as iterações em conjuntos de C iterações contínuas.

A cláusula *schedule (dynamic)* foi escolhida para mitigar o desbalanceamento de carga que ocorre por conta da arquitetura do processador Apple M5. Já que o chip possui 4 supernúcleos e 6 de eficiência, se fizéssemos o escalonamento estático, as threads no núcleo rápido iam terminar primeiro e ficar ociosas. Com o dinâmico a distribuição é feita conforme as threads terminam as iterações.

2.3 Descrição do hardware utilizado nos testes:

MacBook Air

1. Processador: Apple M5
2. CPU de 10 núcleos (4 supernúcleos e 6 núcleos de eficiência)
3. Sistema Operacional: macOS
4. Compilador: GCC 15.2.0

Compilação: `gcc-15 -o filtro -Wall -O3 -fopenmp filtro_gauss.c`

Compilação: `gcc-15 -o filtro -Wall -O3 -fopenmp filtro_gauss_paralelo.c`

2.4 Procedimentos para medição de tempo

Na medição do tempo, foi utilizada a função `omp_get_wtime()` que retorna o *wall clock time*, ou seja, o tempo total decorrido entre o início e o fim de uma tarefa paralela, decorrido em segundos como um valor de precisão dupla.

2.5 Parâmetros utilizados nos testes de desempenho

Variável	Valores Testados
Resoluções das Imagens	512px, 1024px, 2048px, 4096px
Kernel	3x3, 5x5
Número de iterações	100,500,1000
Número de threads	1,2,4,8

Cada teste foi realizado 5 vezes, totalizando 480 testes realizados.

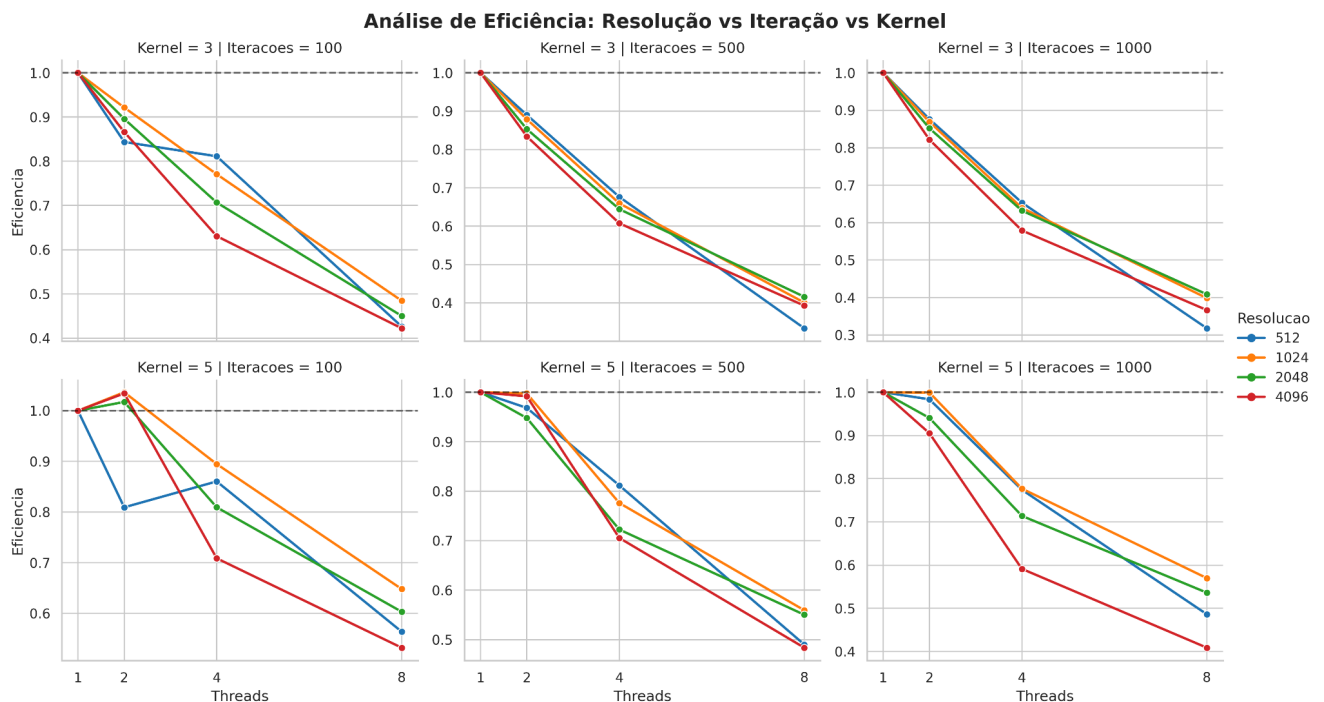
2.6 Métricas utilizadas para avaliação

- Tempo de Execução: O tempo total gasto para processar a imagem;
- Speedup: A razão entre o tempo de execução sequencial e o tempo de execução paralelo. Indica o ganho de velocidade;

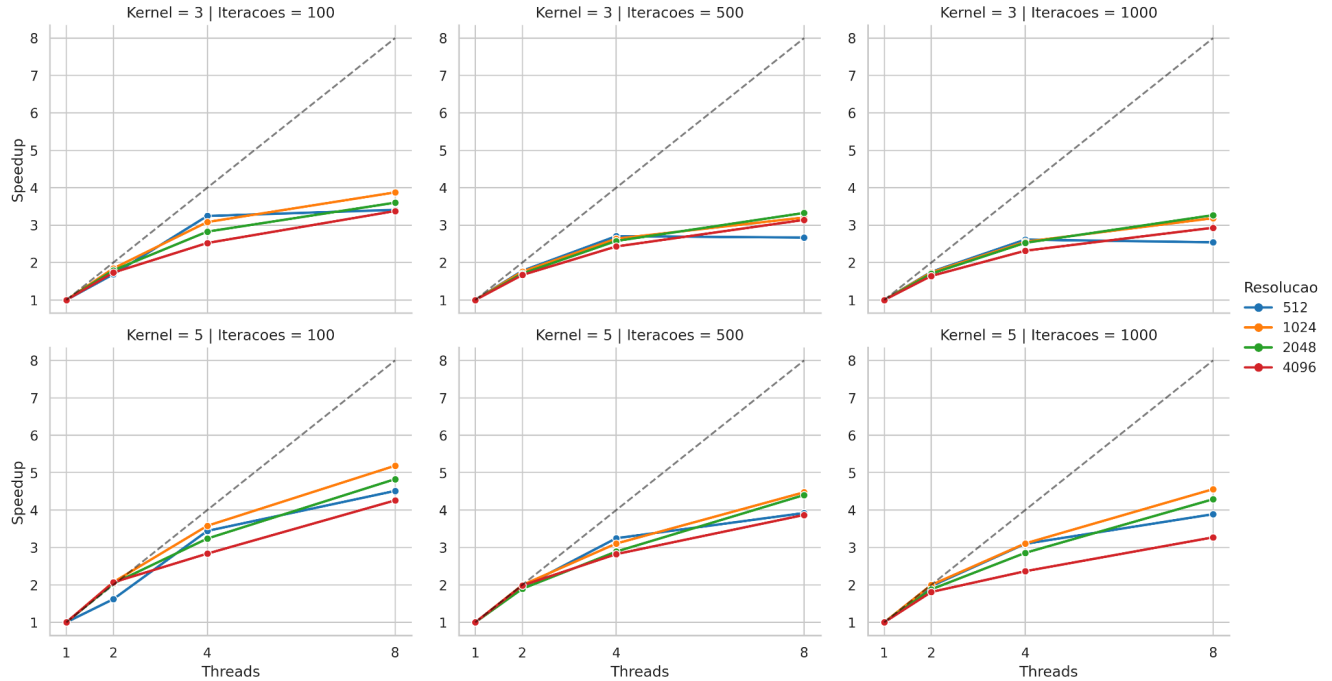
- Eficiência: O Speedup dividido pelo número de processadores/threads. Indica o quão bem os recursos paralelos estão sendo utilizados;
- Fator de Balanceamento de Carga: Quando aplicável (e.g., em MPI com dados desiguais), indica a distribuição do trabalho entre os elementos de processamento;
- Discussão sobre Escalabilidade: Análise de como o desempenho se comporta ao aumentar o número de processadores/threads ou o tamanho do problema.

3. RESULTADOS

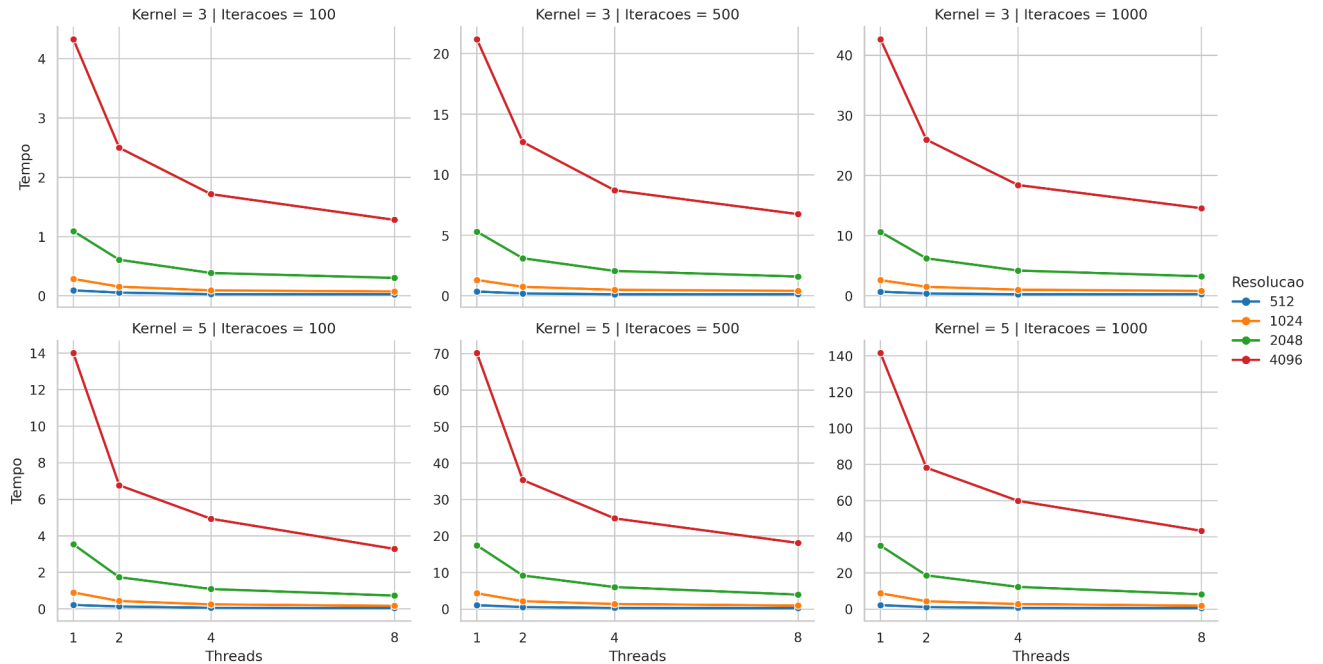
Primeiro utilizamos o código em python com a matriz simples e os parâmetros de exemplo e comparamos com o resultado gerado pelo código serial em C, assim foi possível verificar que eram idênticos. Então fizemos um shell script para executar o código serial e paralelo com diferentes resoluções, tamanho de kernel, quantidade de iterações e número de threads, ao mesmo tempo que comparava a saída da imagem gerada por 8 threads com a gerada em serial, utilizando os mesmos outros parâmetros. Essa comparação foi feita utilizando o *compare* do *imagemagick* tolerando um erro de 1% e com o comando *cmp* que não tolera erros. Dessa forma foi possível verificar que não há diferença nos resultados produzidos, pois em ambos, foi dado como imagens idênticas.



Análise de Speedup: Resolução vs Iteração vs Kernel



Análise de Tempo: Resolução vs Iteração vs Kernel



3.1 Balanceamento de carga e escalabilidade

O principal fator de desbalanceamento identificado foi a arquitetura assimétrica do Apple M5, com 4 núcleos de performance e 6 de eficiência. Em um escalonamento estático, o tempo total seria limitado pelo núcleo mais lento. A utilização da cláusula `schedule(dynamic, 2)` buscou solucionar esse problema ao permitir que os núcleos de

performance processassem uma fatia maior da imagem, reduzindo o tempo de ociosidade das threads mais rápidas.

Quanto à escalabilidade, o algoritmo demonstrou melhor desempenho em cenários de alta intensidade aritmética, como na resolução de 4096x4096 com kernel de 5x5. Isso ocorre porque, com mais cálculos por pixel, o overhead se torna insignificante frente ao tempo de processamento.

4. DISCUSSÃO

4.1 Análise da corretude das implementações

Como nosso teste de corretude não gerou nenhum erro e imagem diferente, podemos concluir que a implementação está correta. A equivalência bit a bit (via *cmp*) e a validação visual (via *compare*) confirmam que o particionamento do laço for com (*dynamic,2*) não introduziu condições de corrida na escrita da imagem de saída e alterações possíveis com a lógica da soma de double.

4.2 Comparação entre as versões sequencial e as paralelas

As versões paralelas apresentaram uma redução drástica no tempo de resposta em relação à execução sequencial, especialmente em resoluções mais altas 4096x4096. Observou-se que o algoritmo de filtro gaussiano possui um alto grau de paralelismo de dados. Entretanto, nota-se que para kernels maiores (5x5), o ganho é mais acentuado, pois a carga computacional por pixel justifica o overhead de criação das threads.

4.3 Impacto do aumento do número de threads

O aumento do número de threads no processamento do filtro gaussiano produz uma redução direta no tempo de execução, embora esse ganho de desempenho não ocorra de forma linear conforme o número de núcleos aumenta. Com base nos dados obtidos, observa-se um *speedup sublinear*, onde o uso de 2 threads gera um ganho de velocidade de aproximadamente 1,74x, enquanto a utilização de 8 threads atinge um pico médio de 3,26x. Esse comportamento demonstra que, embora o paralelismo reduza o tempo total de processamento, existe uma diminuição progressiva na eficiência do sistema, que cai de cerca de 86,8% com 2 threads para apenas 40,7% quando o hardware é levado ao limite de 8 threads.

Em resoluções menores, a intensidade aritmética não é suficiente para compensar o custo de inicialização do paralelismo, enquanto em resoluções maiores o algoritmo escala melhor, embora ainda sofra com a saturação dos recursos de execução. Para o cenário testado, o uso de 4 threads parece representar o ponto de equilíbrio técnico mais equilibrado entre ganho de tempo e consumo de recursos, evitando a queda drástica de eficiência observada ao utilizar a capacidade total do processador.

4.4 Discussão de gargalos e limitações encontradas

Apesar do bom desempenho, em kernels maiores (5x5), a quantidade de acessos à memória por pixel aumenta exponencialmente. Mesmo com a memória unificada do M5, a disputa pelo barramento de memória entre as 8 threads pode gerar latências que limitam o Speedup linear.

Além disso, a escolha de um número pequeno de iterações que são atribuídas a uma thread (dynamic, 2) foi realizada para balancear a existência de núcleos desiguais na máquina, entretanto, acaba introduzindo um custo de gerência. Cada vez que uma thread termina 2 linhas, ela precisa acessar a região crítica do OpenMP para requisitar mais trabalho. Em imagens pequenas, como a de 512x512, esse tempo de espera pode se tornar visível frente ao tempo de cálculo real.

4.5 Sugestões de melhorias

No futuro podemos seguir as seguintes abordagens:

Testar valores intermediários para o dynamic (como 4 ou 8) para encontrar o equilíbrio entre balanceamento de carga e redução de overhead de sincronização.

Utilização de schedule (guided): Esta diretiva inicia com blocos grandes e diminui o tamanho conforme o fim do laço, o que poderia reduzir o overhead inicial mantendo a precisão no final do processamento

5. CONCLUSÃO

5.1 Principais aprendizados do projeto

Foi possível aprender as principais diretivas do openmp, em especial a diretiva for. Também aprendemos como pensar em um problema para paralelizar e onde aplicar corretamente o paralelismo, pois existiam funções, como na criação do kernel que não valia a pena paralelizar, já que era algo muito rápido e simples e ia gastar mais tempo gerenciando as threads do que calculando. Além

disso, entendemos que algumas diretivas podem ser escolhidas não somente de acordo com o problema, mas também em relação a máquina que estamos utilizando, pois como o mac possui 4 super núcleos e 6 de eficiência, quando passa de 4 threads os núcleos de eficiência são utilizados e se utilizássemos for static por exemplo, seria dividido igualmente entre eles e os 4 super núcleos terminaram primeiro, ficando ociosos.

5.2 Considerações sobre desempenho obtido

Os resultados obtidos podem ter sofrido influência do calor, à medida que os testes foram acontecendo o computador foi esquentando e isso pode ter feito o desempenho cair abaixo do que seria esperado na matriz 5x5. Por esse motivo, foram realizados 5 testes com os mesmos parâmetros para garantir um maior grau de exatidão dentro das possibilidades existentes.

6. REFERÊNCIAS

- ORELLANA, Esbel T. Valero. *Aula 06 - Diretivas OpenMP*. Ilhéus: UESC, 2026. Slides de aula da disciplina CET107 - Processamento Paralelo.
- ORELLANA, Esbel T. Valero. *Aula 07 - Diretivas OpenMP*. Ilhéus: UESC, 2026. Slides de aula da disciplina CET107 - Processamento Paralelo.
- OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP 6.0 API Syntax Reference Guide*. [S. l.]: OpenMP ARB, 2024. Disponível em: <https://www.openmp.org/specifications/>. Acesso em: 1 maio 2026.